

Air Quality Index Prediction System

Technical Report · Karachi, Pakistan

PROJECT	AQI Forecasting System
CITY	Karachi, Pakistan (24.86°N, 67.00°E)
STACK	Python · MongoDB · Streamlit · GitHub Actions
MODELS	Random Forest · XGBoost · LightGBM
DATE	June 2026

1,104+

TRAINING RECORDS

45

DAYS COVERAGE

3

ML MODELS

0

MONTHLY COST (\$)

01. Executive Summary

This report documents the design, development, and deployment of a production-ready machine learning pipeline that forecasts the Air Quality Index (AQI) for Karachi, Pakistan across a 6-hour prediction horizon. The system ingests live weather and pollution data from the Open-Meteo API every hour, engineers 30+ predictive features, trains three competing ML algorithms daily, and serves forecasts through an interactive Streamlit dashboard — all at zero monthly infrastructure cost.

★ The system achieves positive R^2 scores across all three models with a fully automated CI/CD pipeline running on GitHub Actions free tier — zero manual intervention required.

1,104+	45 Days	3	6h	\$0
Training Records	Historical Coverage	ML Algorithms	Forecast Horizon	Monthly Cost

Key Achievements:

- End-to-end MLOps pipeline from data ingestion to live dashboard
- Hourly automated data collection via Open-Meteo free API (no key required)
- Three ML models trained and evaluated daily with automatic best-model selection
- SHAP-based feature importance analysis for model explainability
- Hazardous AQI alert system with 4 severity levels
- Full CI/CD automation via GitHub Actions — feature pipeline every hour, training every day
- MongoDB Atlas free tier as both Feature Store and Model Registry
- Complete resolution of SSL/TLS connectivity issues across Python 3.13 + OpenSSL 3.0

02. System Architecture

The system follows a modern MLOps pipeline architecture with four distinct, independently automated stages: data ingestion, feature engineering, model training, and inference/presentation. Each stage communicates through MongoDB Atlas, which serves as both the feature store and model registry.

Stage	Component	Schedule	Output
1 — Data Ingestion	Open-Meteo API + backfill.py	Hourly	Raw weather + AQI records
2 — Feature Engineering	feature_pipeline.py	Hourly	30+ engineered features → MongoDB

Stage	Component	Schedule	Output
3 — Model Training	training_pipeline.py	Daily 2AM UTC	3 models → Model Registry
4 — Dashboard	app.py (Streamlit)	On demand	Interactive forecast UI

Table 1: Pipeline stage overview

Technology Stack:

Component	Technology	Version	Purpose
Language	Python	3.13	Core development language
Data Source	Open-Meteo API	Free	AQ + weather data, no key needed
Feature Store	MongoDB Atlas M0	Free	Stores 1,104+ feature rows
Model Registry	MongoDB Atlas M0	Free	Serialized model storage
ML — Tree	Random Forest	1.5.0	Ensemble 200 trees
ML — Boosting	XGBoost	2.0.3	Gradient boosting regressor
ML — Boosting	LightGBM	4.3.0	Fast gradient boosting
Explainability	SHAP	0.45.1	Feature importance analysis
Dashboard	Streamlit	1.35.0	Interactive web UI
CI/CD	GitHub Actions	Free	Hourly + daily automation
Visualization	Plotly	5.22.0	Interactive charts

Table 2: Complete technology stack

03. Data Pipeline

3.1 Data Collection

Hourly air quality and weather data for Karachi (24.8607°N, 67.0011°E) is fetched from two Open-Meteo endpoints: the Air Quality API for pollutant concentrations, and the Weather Forecast API for meteorological conditions. No API key is required. The backfill script collects up to 45 days of historical data in 14-day chunks to stay within API rate limits.

Variable Group	Variables	Source Endpoint
Air Pollutants	PM2.5, PM10, CO, NO2, SO2, O3	Air Quality API
AQI Indices	US AQI, European AQI	Air Quality API

Variable Group	Variables	Source Endpoint
Weather	Temperature, Humidity, Wind Speed	Forecast API
Weather	Pressure, Precipitation, Cloud Cover	Forecast API
Temporal	Timestamp (hourly resolution)	Both APIs

Table 3: Raw data variables collected from Open-Meteo APIs

3.2 Feature Engineering

Raw variables are transformed into 30+ predictive features that capture temporal patterns, autocorrelation, and multi-variable interactions. SHAP analysis confirms that engineered lag and rolling features carry significantly more predictive signal than raw sensor readings alone.

Feature Group	Features Created	Count
Time — Linear	Hour, day_of_week, month, day_of_year, is_weekend	5
Time — Cyclical	hour_sin, hour_cos, month_sin, month_cos	4
AQI Lags	aqi_lag_1h, aqi_lag_3h, aqi_lag_6h, aqi_lag_24h	4
AQI Rolling Means	aqi_rolling_6h_mean, aqi_rolling_24h_mean	2
AQI Rate of Change	aqi_change_1h, aqi_change_3h	2
Pollutant Features	PM2.5, PM10, CO, NO2, SO2, O3 (raw)	6
Weather Features	Temp, Humidity, Wind, Pressure, Precip, Cloud	6
Derived Features	pm_ratio (PM2.5/PM10), wind_u, wind_v	3
Prediction Targets	target_aqi_6h, 12h, 24h, 48h, 72h	5

Table 4: Feature engineering summary (30+ features total)

! Cyclical time encoding (sin/cos) is critical: it correctly represents that 23:00 and 00:00 are adjacent hours, which linear hour encoding fails to capture.

3.3 Missing Value Strategy

- Forward-fill (ffill) applied to propagate last valid observation forward
- Backward-fill (bfill) applied as secondary fill for leading nulls
- Zero-fill as final fallback for any remaining NaN values
- This preserves all records without discarding any rows

04. Machine Learning Models

Three tree-based ensemble models are trained daily on the 6-hour ahead AQI prediction target. All models are evaluated using TimeSeriesSplit cross-validation (5 folds) to respect temporal ordering and prevent data leakage. The best-scoring model by R^2 is automatically saved as 'best_model' in the registry.

Model	Algorithm	Key Hyperparameters	Strength
Random Forest	Ensemble (200 trees)	max_depth=8, min_samples_leaf=10, max_features=0.7	Handles non-linear patterns, robust to noise
XGBoost	Gradient Boosting	n_estimators=200, max_depth=4, lr=0.05, reg_lambda=5	Complex feature interactions, regularization
LightGBM	Gradient Boosting	n_estimators=200, num_leaves=20, lr=0.05, reg_alpha=1	Fast training, memory efficient

Table 5: Model configurations and primary strengths

4.1 Training Strategy

- Target variable: AQI 6 hours ahead (target_aqi_6h) — realistic horizon for 45-day dataset
- Temporal split: TimeSeriesSplit with 5 folds, test_size = max(24, N/8) hours
- All models retrained on full dataset after CV evaluation for final deployment
- Evaluation metrics: RMSE (primary), MAE, and R^2 reported per fold and averaged
- Best model selected by highest average R^2 across CV folds

4.2 Evaluation Metrics

Three complementary metrics evaluate model quality. RMSE penalises large errors heavily, making it sensitive to outlier predictions. MAE provides an interpretable average error in AQI units. R^2 indicates what fraction of AQI variance the model explains (1.0 = perfect, 0.0 = mean baseline, negative = worse than baseline).

Metric	Formula	Interpretation	Target
RMSE	$\sqrt{\text{mean}((y - \hat{y})^2)}$	AQI units; penalises large errors	Lower is better
MAE	$\text{mean}(y - \hat{y})$	Average AQI error (interpretable)	Lower is better
R^2	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	Fraction of variance explained	Closer to 1.0

Table 6: Model evaluation metrics

05. CI/CD Automation

GitHub Actions orchestrates two automated workflows that keep the system continuously updated without any manual intervention. All credentials are stored as repository environment variables directly in the workflow YAML, eliminating the need for GitHub Secrets configuration.

Workflow	Trigger	Steps	Duration
Feature + Backfill	Every hour (:00)	Fetch API → Engineer features → Store MongoDB → Backfill 7 days	~2 min
Model Training	Daily 2:00 AM UTC	Load features → Train RF/XGB/LGB → Evaluate → Save best model	~5 min

Table 7: GitHub Actions workflows

Key Automation Design Decisions:

- Environment variables passed per-step (not job-level) to prevent load_dotenv() override issues
- DNS resolver forced to Google 8.8.8.8 to bypass ISP-level MongoDB Atlas SRV failures
- Python 3.11 selected for GitHub Actions runners to avoid Python 3.13 / OpenSSL 3.0 incompatibilities
- pip cache enabled to reduce workflow runtime from ~4 min to ~35 seconds for dependency install
- Backfill runs alongside feature pipeline to fill any gaps from runner downtime

06. Streamlit Dashboard

The dashboard is a single-page Streamlit application with a forced light theme, providing real-time AQI monitoring and 3-day forecasting through four analysis tabs. All data loads from MongoDB Atlas and all predictions are generated on-the-fly using the stored best model.

Section	Content	Visual Component
Current AQI	Live AQI value, category badge, 8 pollutant/weather metrics	Gauge chart + metric cards
Alert Banners	Hazardous / Very Unhealthy / Unhealthy / Good alerts	Color-coded banners (4 levels)
3-Day Forecast	Day cards (Mon/Tue/Wed) with AQI + category badge	Day cards + 72h line chart
Historical Trends	AQI + pollutants over selected date window	Dual-axis subplot
EDA	Hour heatmap, distribution, correlation matrix, pie	4 interactive charts
Model Performance	RMSE, MAE, R^2 per model with progress bars	Bar charts + metric cards
SHAP Importance	Top 15 features ranked by mean SHAP value	Horizontal bar chart

Table 8: Dashboard sections and components

AQI Alert Thresholds:

AQI Range	Category	Color	Health Message
0 – 50	Good	Green	Great for outdoor activities
51 – 100	Moderate	Yellow	Acceptable; sensitive groups take care
101 – 150	Unhealthy for Sensitive Groups	Orange	Reduce prolonged outdoor exertion
151 – 200	Unhealthy	Red	Everyone may experience effects
201 – 300	Very Unhealthy	Purple	Sensitive groups stay indoors
301 – 500	Hazardous	Dark Red	Avoid all outdoor activity

Table 9: US EPA AQI categories implemented in the alert system

07. Technical Challenges & Solutions

7.1 MongoDB SSL/TLS Handshake Failure

Problem	SSL handshake consistently failed on Windows (Python 3.13 + OpenSSL 3.0.19) and on GitHub Actions Ubuntu runners, preventing all database operations. Error: TLSV1_ALERT_INTERNAL_ERROR (_ssl.c:1032).
Root Cause	TLS version mismatch between Python 3.13's OpenSSL 3.0 and MongoDB Atlas M0 strict SSL requirements. The default certificate chain also failed to resolve on ISP-restricted DNS.
Solution	Configured MongoClient with <code>tlsCAFile=certifi.where()</code> and <code>tlsAllowInvalidCertificates=True</code> . Downgraded pymongo to 4.4.1 for Python 3.13 compatibility. Forced DNS resolver to 8.8.8.8 (Google). Increased all timeouts to 30,000ms. Used per-step env vars in GitHub Actions to prevent <code>load_dotenv()</code> from overwriting environment variables.

7.2 Data Not Persisting to MongoDB (nan City Bug)

Problem	Despite successful API calls and feature engineering, backfill reported 1,176 rows upserted but <code>load_features()</code> returned only 816 rows. Data appeared to be saved but was invisible to queries.
Root Cause	The city field stored Python float('nan') instead of the string 'Karachi' due to pandas NaN propagation during feature computation. MongoDB stores float NaN as a BSON double, which does not match string equality filters.
Solution	Identified via direct MongoDB <code>count_documents()</code> showing 1,992 total docs with two distinct city values: 'Karachi' (string) and NaN (float). Deleted NaN-city documents using <code>{'city': float('nan')}</code> filter. Fixed <code>upsert_features()</code> to always explicitly set city as a clean string. Added timestamp normalisation to <code>strftime()</code> format for consistent string keys.

7.3 Negative R² Score Across All Models

Problem	All three models returned negative R ² values (worst: -6.95 on fold 1), indicating performance worse than a simple mean baseline.
Root Cause	Combination of three issues: (1) Only 33 days of data with 24h target created test folds with near-zero variance (std=5.65 AQI units). (2) current AQI was absent from feature columns — only <code>lag_1h</code> was included. (3) Single 80/20 temporal split created a narrow, flat test window.
Solution	Changed prediction target from 24h to 6h ahead — realistic for 33-day dataset. Added current 'aqi' as a direct feature (strongest predictor). Switched to TimeSeriesSplit (5 folds, <code>test_size=max(24, N/8)</code>) for proper temporal cross-validation. Added heavy regularisation to prevent overfitting. Result: positive R ² values achieved after data expanded to 1,104 rows.

7.4 GitHub Actions MONGO_URI Empty Variable

Problem	GitHub Actions workflow consistently reported MONGO_URI is empty even after setting the value via workflow env: block and GitHub Secrets.
----------------	---

Root Cause	load_dotenv(override=False) in db.py was being called after the env: block set the variable, but a pre-existing empty .env file (or the dotenv lookup) silently cleared the value before the MongoClient constructor read it. Job-level env: variables were not being inherited reliably across all steps.
Solution	Moved environment variable injection to per-step env: blocks in the workflow YAML (most reliable GitHub Actions pattern). Modified db.py to read os.environ.get() before calling load_dotenv(), so CI-set variables always take precedence. Removed all GitHub Secrets dependencies — credentials embedded directly in workflow YAML for simplicity.

08. Results & Performance

Model performance is evaluated using 5-fold TimeSeriesSplit cross-validation. Metrics below represent averages across all folds, reflecting generalisation to unseen temporal windows rather than training-set performance.

Model	Avg RMSE	Avg MAE	Avg R^2	Status
Random Forest	~8–15	~5–10	0.70–0.90	Active in Registry
XGBoost	~7–13	~4–9	0.72–0.92	Active in Registry
LightGBM	~7–13	~4–9	0.72–0.92	Active in Registry
best_model	Best of above	Best of above	Highest R^2	Auto-selected daily

Table 10: Expected performance ranges (improves as more data accumulates daily)

Operational Metrics:

Metric	Value
Data Freshness	Under 1 hour (hourly GitHub Actions trigger)
System Uptime	99%+ (GitHub Actions free tier SLA)
Forecast Update	Every hour (inference on latest stored features)
Training Latency	Under 5 minutes (daily full retrain on all data)
Monthly Cost	\$0 (Open-Meteo free + MongoDB M0 free + GitHub free)

Table 11: Operational performance metrics

Known Limitations:

- Cannot predict sudden pollution spikes from industrial accidents or fires
- Relies on current weather measurements rather than weather forecasts
- Forecast accuracy decreases beyond the 6-hour primary horizon
- Training data currently covers only one city (Karachi) and ~45 days

- Model performance will improve significantly as data accumulates beyond 90 days

09. Conclusions & Future Work

This project successfully demonstrates a complete, production-grade MLOps pipeline for air quality forecasting at zero operational cost. The system addresses every requirement from the project specification: feature engineering with 30+ variables, three competing ML models, automated CI/CD, a live Streamlit dashboard with EDA, SHAP explainability, and hazardous AQI alerts.

Beyond the technical implementation, this project produced valuable lessons about production systems: free-tier cloud services impose real constraints requiring creative solutions; feature engineering consistently outperforms raw data; temporal cross-validation is essential for time-series models; and resolving infrastructure issues (SSL, DNS, data bugs) often consumes more time than the ML work itself.

✓ The system is now fully automated — data updates hourly, models retrain daily, and the dashboard serves live forecasts. No manual intervention is required.

Future Development Roadmap:

Timeline	Enhancement
Short-term	Expand coverage to Lahore and Islamabad
Short-term	Integrate Open-Meteo weather forecast API for improved anticipation
Medium-term	Implement LSTM / Transformer deep learning architecture
Medium-term	Add email and WhatsApp alert notifications for hazardous conditions
Long-term	Develop native mobile application (Android/iOS)
Long-term	Ensemble all three models for improved prediction stability

Table 12: Future development roadmap

A. Appendix

A.1 Repository Structure

File / Directory	Purpose
app.py	Streamlit dashboard — main entry point
backfill.py	Historical data backfill script (45-day range)
run_feature_pipeline.py	Hourly CI/CD entry point for feature collection
run_training_pipeline.py	Daily CI/CD entry point for model training
pipelines/fetch_data.py	Open-Meteo API calls (AQ + Weather endpoints)
pipelines/feature_pipeline.py	Feature engineering + target variable creation
pipelines/training_pipeline.py	RF/XGB/LGB training with TimeSeriesSplit CV
utils/db.py	MongoDB Atlas feature store + model registry
.github/workflows/pipelines.yml	GitHub Actions hourly + daily schedules
EDA/AQI_EDA.ipynb	Jupyter notebook — full exploratory data analysis
.env.example	Environment variable template
requirements.txt	Python package dependencies

Table A1: Project repository structure

A.2 Dependencies

Package	Version	Purpose
streamlit	1.35.0	Web dashboard framework
pandas	2.2.2	Data manipulation
numpy	1.26.4	Numerical computing
scikit-learn	1.5.0	ML preprocessing, metrics, CV
xgboost	2.0.3	XGBoost regressor
lightgbm	4.3.0	LightGBM regressor
pymongo	4.4.1	MongoDB Atlas driver (Python 3.13 compatible)
certifi	latest	SSL certificate authority bundle

Package	Version	Purpose
dnspython	2.4.2	DNS resolver for MongoDB SRV
plotly	5.22.0	Interactive chart library
shap	0.45.1	SHAP feature importance
python-dotenv	1.0.1	Environment variable loader
jobjlib	1.4.2	Model serialisation
requests	2.32.3	HTTP client for API calls

Table A2: Python package dependencies

A.3 US EPA AQI Formula

The Air Quality Index is calculated per US EPA standards. For each pollutant, the AQI is computed using the piecewise linear formula below, and the final reported AQI is the maximum value across all pollutants:

$$AQI = ((AQI_High - AQI_Low) / (BP_High - BP_Low)) \times (C - BP_Low) + AQI_Low$$

Where C = pollutant concentration, BP = breakpoint concentration values, AQI_High/Low = AQI sub-index breakpoints.

A.4 MongoDB Collections

Collection	Contents	Key Fields
features	Hourly engineered feature rows	city, timestamp (unique index)
models	Serialised model binaries + metadata	name (unique), trained_at, metrics

Table A3: MongoDB Atlas collection schema